



PDF Download
3696630.3728575.pdf
25 March 2026
Total Citations: 1
Total Downloads: 827

Latest updates: <https://dl.acm.org/doi/10.1145/3696630.3728575>

SHORT-PAPER

Analyzing Evolution of Microservice-based Systems

TOMAS CERNY, The University of Arizona, Tucson, AZ, United States

GABRIEL GOULIS, The University of Arizona, Tucson, AZ, United States

SAMANTA PERRY, The University of Arizona, Tucson, AZ, United States

MALIA EDMONDS, The University of Arizona, Tucson, AZ, United States

AMR S. ABDELFAHATTAH, The University of Arizona, Tucson, AZ, United States

MATTEO ESPOSITO, University of Oulu, Oulu, North Ostrobothnia, Finland

[View all](#)

Open Access Support provided by:

The University of Arizona

University of Oulu

Published: 28 July 2025

Citation in BibTeX format

FSE Companion '25: 33rd ACM
International Conference on the
Foundations of Software Engineering
June 23 - 28, 2025
Trondheim, Norway

Conference Sponsors:
SIGSOFT

Analyzing Evolution of Microservice-based Systems

Tomas Cerny
University of Arizona
Tucson, AZ, USA
tcerny@arizona.edu

Malia Edmonds
University of Arizona
Tucson, AZ, USA
maliaedmonds@arizona.edu

Alexander Bakhtin
Oulu University
Oulu, Finland
alexander.bakhtin@oulu.fi

Gabriel Goulis
University of Arizona
Tucson, AZ, USA
ggoulis@arizona.edu

Amr S. Abdelfattah
University of Arizona
Tucson, AZ, USA
amrelsayed@arizona.edu

Valentina Lenarduzzi
Oulu University
Oulu, Finland
valentina.lenarduzzi@oulu.fi

Samantha Perry
University of Arizona
Tucson, AZ, USA
samnperry@arizona.edu

Matteo Esposito
Oulu University
Oulu, Finland
matteo.esposito@oulu.fi

Davide Taibi
Oulu University
Oulu, Finland
davide.taibi@oulu.fi

Abstract

Most leading technology companies are already on the cloud and continue investing in cloud-native systems built on microservices. Despite the trajectory, we lack guidance and best practices when coping with the evolvability of microservice-based systems. In a microservice architecture, applications are structured as a collection of loosely coupled, independently deployable services, each responsible for a specific functionality. However, despite the isolated microservice codebases managed by distinct teams, dependencies exist across microservices. When changes to the microservice code are made, we risk impacting other services with the potential for breakage when these get deployed. This compromises resilience and undermines stability throughout the distributed evolution. Unfortunately, we lack effective mechanisms to mitigate breaking changes when they are introduced. Our tool performs a comprehensive distributed codebase analysis based on robust AST-parsing tailored to component-based development typical for microservices. Such state-of-the-art static analysis is a promising instrument to effectively detect breaking changes or anti-patterns. Given the plethora of platforms, our demonstration focuses on Java and invites contributors to broaden platform support, offering unified intermediate representation to facilitate analysis with reusable search rules.

Website: cloudhubs.github.io/cimet/ | Video tutorial: youtu.be/mKWueuln_80

CCS Concepts

• **Computer systems organization** → **Distributed architectures**; • **Software and its engineering** → **Software evolution**.

Keywords

Microservices, Change Impact, Evolution, Anti-patterns

ACM Reference Format:

Tomas Cerny, Gabriel Goulis, Samantha Perry, Malia Edmonds, Amr S. Abdelfattah, Matteo Esposito, Alexander Bakhtin, Valentina Lenarduzzi, and Davide Taibi. 2025. Analyzing Evolution of Microservice-based Systems. In *33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion '25)*, June 23–28, 2025, Trondheim, Norway. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3696630.3728575>

1 Introduction

Microservices are no longer a question for technological companies; the question is how to ensure that decentralized teams making independent microservice changes do not break the system services that participate in distributed tasks. Newspapers inform us about unfortunate bank systems [21, 22] leaving clients with declined credit cards at gas stations because of breaking system updates or clients seeing someone else's account in their web interface. All these can be potential consequences of poorly managed co-change where developers did not realize change impact on other microservices.

What can we do to fix this? Companies can double the development teams to review changes of other teams, which is a never-ending process, or we can ignore the problem until it gets into production and depend on tracing and prompt root cause analysis [28]. If you agree, this is not the way to go; the community has to resort to just-in-time analysis. However, microservices cannot employ common static analyzers because (1) they use component-based development, and (2) one has to cope with decentralized codebases that lack explicit interconnection.

Other aspects might exist. Some may say we need to deal with system polyglots [25]. While it is a very popular topic in scientific forums, after interviewing many tech software architects in the United States, it seems to be a marginal case in practice. When we accept static analysis direction, we can observe research using regular expressions [18, 30] to match component properties from the source code and use them for their custom rules in architecture assessment. Yet, one must agree that robust AST code parsing is a more robust approach to invest in with a long-term lifespan.

This paper introduces a new tool to spin off coordinated community efforts for microservice change assessment. It emerges from the



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

FSE Companion '25, Trondheim, Norway

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1276-0/2025/06

<https://doi.org/10.1145/3696630.3728575>

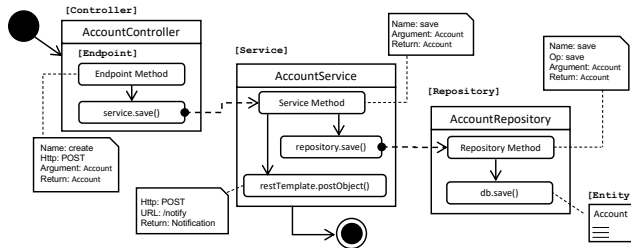


Figure 1: Component Call Graph [14] illustrating Account-Controller endpoint connected to an AccountService and AccountRepository that manages and initiates remote call while using an Account data entity.

foundation of multiple prior prototypes for software architecture reconstruction and anti-pattern detection [4, 5, 8, 11, 17, 19, 23, 26, 32]. The tool is designed “to deal with evolving system aspects” and uses AST-parsing tailored to component-based code. With its intermediate representation that emphasizes components, we can (1) welcome various microservice development platforms and (2) identify and manage microservice dependencies. Consequently, with such an abstract model of the overall system, we demonstrate sample change impact and anti-pattern detection that stand on the system abstraction and thus can be improved or reused by others.

We demonstrate our tool in action on eight open-source projects with over 700 commits and illustrate its findings in terms of identified issues over the projects’ evolution in their git history.

Paper Structure: Section 2 provides background for our work, and Section 3 presents the tool design. Section 4 presents the initial tool empirical validation. Finally, we draw conclusions in Section 5.

2 Background

The evolvability of a software system refers to its ability to adapt, extend, and improve over time in response to changing requirements, technologies, or environments while maintaining quality and minimizing costs. It is a crucial aspect of software engineering, as software must remain functional and relevant in dynamic and often unpredictable contexts. Evolution is manifested by a change, in our case, a change in source code.

There are just-in-time tools that work the charm in monolithic systems to cope with change impact analysis and anti-patterns [1]. These tools work by code AST-parsing and traversals for rules that indicate problems or smells. For change impact analysis [7, 20] tools track control and data dependencies, and what is typically built is a system dependency graph, which is then used to track changed code parts and their propagation through the system dependency graph traversal. To cope with system evolution, statistics are tracked of anti-patterns across versions; alternatively, change impact analysis across code updates can facilitate code reviews [18, 30, 31].

With microservices, we deal with the aspect of multiple individual, self-contained codebases. However, there exist dependencies across microservices [3, 15, 27]. These must be managed, as these are critical to the purpose of change analysis [20].

Apart from this, we must accept that there is no vanilla code for which most static analysis tools were designed. Understanding the architecture of microservices requires an in-depth assessment of the components making up these systems and inspecting how

they interact [10]. One should understand the details of individual elements in every microservice and understand microservice interconnection. For instance, Fig. 1 illustrates a slice of 3 layered microservice architecture that shows an endpoint of a particular controller that uses service and repository components and operates with an entity that defines persistent data.

In prior works on software architecture reconstruction [14], we used the component call graphs constructed from component-tailored AST parsing as an intermediate representation. Next, we used managed dependencies to connect microservice intermediate representations over (1) recognized remote calls to system endpoints and (2) through data similarity; the result is a connected system abstraction model for performing architecture analysis. This, however, lacks a temporal perspective of system evolution.

To assess the quality and efficiency of a system’s architecture, we typically rely on architectural quality and performance metrics [2, 9, 12]. Along metrics such as coupling, cohesion, and complexity [10], to comprehensively measure the reliability of architecture, we also identify anti-patterns—poor design solutions that degrade system architecture through inefficiency or maintainability issues [13].

To recognize anti-patterns for microservice-based systems, we constructed and unified a large catalog of 5 categories and 58 anti-patterns. Obviously, the increase in anti-patterns and the absence of intended design solutions indicates an issue.

For innovation, we could focus on a situation where change leads to potential breakage, like changing an endpoint used by someone else or changing data used as an endpoint parameter, not articulating it to others. Finally, architectural changes are modifications across the system over time [2, 15]. Architectural changes may involve updating service interfaces, changing service dependencies, or even introducing new services [29]. Changes are consequences of the system’s evolving business requirements, new technologies, or performance improvement [24]. Since microservices are highly interconnected, small changes can also cause a high impact across the system and lead to potential unintended consequences, like increased coupling or degradation of architecture [10, 24].

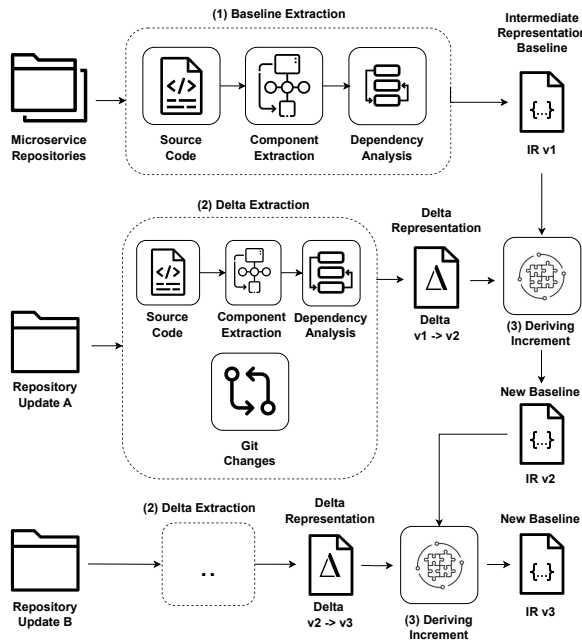
3 CIMET: Change Impact Microservice Evolution Tool

The core concept of our tool uses incremental software architecture reconstruction (iSAR) [16]. The process initiates with a one-time complete distributed codebase analysis to uncover a baseline abstract model of system architecture across all microservices. This process has been described in the background, using component-tailored AST parsing [25] and producing an interconnected component call graph (i.e., JSON format) [14]. This is highlighted in Fig. 2 as a baseline extraction. Next comes the foundational innovation.

Consequent code change can come from any decentralized codebase in the form of a commit or a pull request. Such a change determines what we refer to as a delta, which is a subject of just-in-time analysis. A delta encapsulates a specific change in a single codebase. This has a two-fold consequence: First, to maintain the system abstraction baseline up-to-date, the delta is mapped to the baseline to determine the new baseline increment (see Fig. 2). This serves the temporal perspective of evolution. Our process enables the execution of baseline increments in a decentralized perspective.

Table 1: Description of implemented Anti-patterns and Code Quality Metrics

	Name	Description
Anti-patterns	Greedy Microservice	The Microservice is dependent (outgoing dependencies) over some threshold value.
	Hub-Like Microservice	The Microservice is depended on (incoming dependencies) over some threshold value.
	Service Chain	A number of Microservice dependencies above some threshold value form a linear series of calls.
	Wrong Cuts	A Microservice does not leverage data access or storage.
	Cyclic Dependency	A number of Microservice dependencies form a cycle.
	Wobbly Service	A Microservice endpoint
	No Healthcheck	A Microservice does not leverage health checks.
	No API Gateway	A Microservice does not leverage an API Gateway.
Metrics	Absolute Importance of the Service (AIS)	The number of dependents of a Microservice, invoking at least one operation.
	Absolute Dependency of the Service (ADS)	The number of microservice dependencies a Microservice invokes at least one operation on.
	Absolute Criticality of the Service (ACS)	The product of the AIS and the ADS.
	Service Coupling Factor (SCF) (<i>Graph density</i>)	The quotient of the number of unidirectional dependencies and the total number of potential dependencies.
	Service Interdependence of System (SIY)	The number of interdependent services in the system.
	Structural Coupling (SC)	A measure of the number of dependencies between two services
	Strict Interface Data Cohesion (SIDC)	The similarity of the parameters data-types between two services
	Strict Service Implementation Cohesion (SSIC)	The cohesion of a given service (s) based on the cohesiveness of the operations exposed in its interface
	Lack of Message Level Cohesion (LOMLC)	The cohesion of a services operations

**Figure 2: Incremental Software Architecture Reconstruction process [16].**

For instance, the initial baseline might correspond to the deployed system version rather than the current development branches, and thus, microservice teams might reason about staging changes.

The second consequence comes with the iterative process format. Upon each delta integration, the just-in-time change analysis can be performed in the context of the whole system (i.e., metric calculations, anti-pattern detection, impact analysis). Our analysis operates on the system abstraction model (i.e., JSON), with access to the delta, prior, and new baseline versions. It is decoupled from the iSAR process to resort away from platform specificity.

The tool’s logical architecture is divided into three packages: Intermediate, Delta, and Detection. The *Intermediate package* copes with AST-parsing, baseline extraction, and up-to-date maintenance. Moreover, it produces a sequence of hierarchical JSON structures representing the abstract interconnected system model.

The *Delta package* extracts delta in the changed source code. The Delta package operations are iterative, i.e., traversing the locally cloned repository leveraging the git adapters. The delta files come in a sequence, interweaving the abstract system models and serving as baselines at given times. The Delta file contains all the added/modified/removed code structures from version to version that are mapped to the new baseline.

The *Detection package* performs impact analysis on top of the results from the given version transition. Unaware of the originating source code, its logic comes from various algorithms, metrics, and rules. The tool supports code quality metrics and anti-pattern details in Table 1. These are reported for every commit in the repository history.

4 Initial CIMET Empirical Validation

The evaluation of our tool is performed on eight benchmark systems selected from the published dataset [6] for the following properties: using Java Spring and having more than one dependency between services. These properties were manually validated when selecting projects. Six projects were sourced from this microservice dataset, and two were identified online by searching public repositories. Table 2 lists these projects.

The design of our evaluation leverages the open-source GitHub repository for comparing delta output alongside physical code changes. The process starts with the tool running unobstructed to get the output for validation. For testing and validation purposes, we maintain lists of all intermediate representations (interconnected system abstractions) and delta output files.

Our initial approach to validation was to ensure the Delta output was accurate; this was done in two ways. We validated that the reconstructed system computed via delta extractions matched a

Table 2: Microservice projects used in the study

Name (http://github.com/..)	#Commits	Description
Springboot-Microservice	7	Personal Project
spring-cloud-movie-recommendation	6	Personal Project
spring-boot-microservices-workshop	12	Educational Project
microservices-basics-spring-boot	46	Educational Project
java-microservice	140	Educational Project
train-ticket	323	Educational Project
spring-microservices	26	Personal Project
sample-spring-microservices-new	181	Educational Project

directly parsed system at the final commit. This confirmed that the reconstruction process was accurate and captured all change information. Further, we spot-checked a random sample of delta files with physical code changes presented in GitHub repositories. This random sample (e.g., 50 files) was thoroughly examined to verify that the delta output accurately represented a 1:1 mapping of the physical code changes. As a result, we ensure that all collected data accurately represents the system history and precisely aligns with the changes made in the system source code.

With validating captured metrics and anti-patterns, we performed a systematic validation process on all project outputs. During this procedure, we manually validated with two independent reviewers that metrics and anti-pattern output were accurate. For each system and each commit, two researchers independently identified anti-patterns present and computed metric values to find the ground truth. In the case of disagreement, a third reviewer would be used for a tiebreak. For the "train-ticket" project, we took a subset based on a confidence interval of 95%, leaving us with 176 commits to validate. For all other projects, we validated the entire system history. We outlined an error rate of 5% for the validation process.

4.1 Results

Eight total projects were considered when undergoing validation of our study. Of the eight projects, all but one found consensus among two independent reviewers for all commits. There were no disagreements during the validation process, so a third reviewer was not required for any of the project's validation.

The validation process was restarted once when a bug regarding dependency matching was identified and fixed. Further, a lapse in support for query parameters was resolved before performing the second and final validation effort. In this final validation effort, we found consistent results regarding what we computed and what the tool identified. The validation process yielded below the 5% error rate outlined in our study design for all remaining projects.

To allow the verifiability and replicability of our findings and the tool usage, we published the source code and the data on Zenodo¹.

4.2 Discussions

Our evaluation provides insights into the architectural evolution of considered microservice projects in terms of anti-patterns and code quality metrics that reflect the quality of system architecture at a given version.

Figure 3 illustrates the evolution trends of anti-patterns and metrics detected throughout the history of the train-ticket project. In all projects, we were able to identify degraded architecture in the presence of many anti-patterns over time, low rates of cohesion, and high rates of coupling. Most commonly seen in output were instances of Wrong Cuts, No Health Checks, and no API gateway. These anti-patterns affect service maintainability and the ability of the overall system to scale. For metrics, we generally saw low rates of coupling, such as structural coupling, but also low rates of cohesion in metrics, such as strict service implementation cohesion. This means microservice implementations are scattered in responsibility decreasing maintainability of the service and system overall. We observed that as microservices become less cohesive in

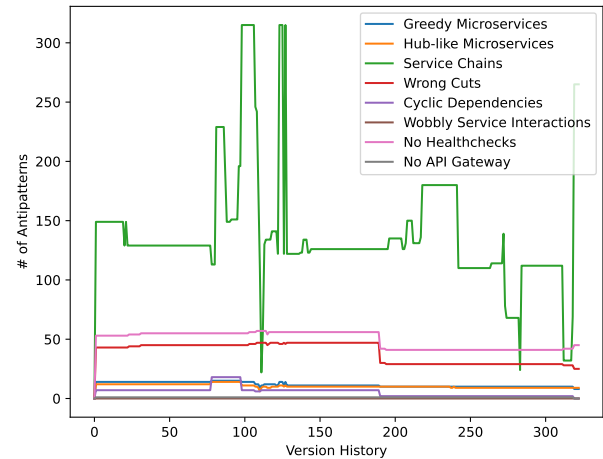


Figure 3: Train Ticket Anti-patterns Over Versions

implementation, metrics such as their Absolute Dependency of the Service rise as they fulfill more responsibilities. These indicators provide justification for our tool as we are able to draw data-led insights on the state of the system over time.

4.3 Threats to Validity

We categorized the threats in Construct, Internal, External, and Conclusion validity following established guidelines [33].

Construct Validity Our tool is based on the assumption that microservice code is accessible and follows a component-based practice from which it extracts component models serving as intermediate representations. We based on established anti-pattern or metric definitions and our interpretation and was validated by at least two senior personnel. Our tool did not consider dynamic URL paths when parsing microservice interconnections.

Internal Validity Our tool is based on static analysis, which is threatened by approximation, and identified anti-patterns or metrics that might be slightly deviated; we assessed the finding manually to limit this effect.

External Validity: To minimize the specialization, we selected eight projects for evaluation, some commonly used in literature for evaluation. While we draw results for Java projects, other platforms might present different trends. Two researchers were involved in limiting bias.

Conclusion Validity: Our conclusions rely on the specific accuracy metrics chosen, and there may be other aspects or dimensions of performance that we did not consider. To address this potential limitation, we selected metrics from recent related studies [2, 9, 10, 12].

5 Conclusions and Future Works

Without supporting tools aiding the evolvability of microservice systems, improvements to the process are hard to introduce. Our tool helps with change analysis to better understand the just-in-time impact. It tracks indicators of architectural quality through the identification of anti-patterns and code quality metrics. We demonstrated the tool application on eight benchmark systems tracking their commit history, where anti-patterns, cohesion, and

¹Zenodo dataset <https://doi.org/10.5281/zenodo.13914563>

coupling metrics are observed. Apart from the open source tool, which we hope peers can help improve, we also share reproduction package¹ with a broad set of artifacts for the community related to our findings across selected projects. In future work, we plan to conduct a large empirical study and extend tool capabilities to capture additional metrics and run on different platforms.

Acknowledgement

This material is based upon work supported by the National Science Foundation under Grant No. 2409933. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the National Science Foundation.

References

- [1] [n. d.]. Code Quality, Security and Static Analysis Tool with SonarQube | Sonar. <https://www.sonarsource.com/products/sonarqube/>. (Accessed on 02/05/2024).
- [2] 2023. Microservices Adoption in 2020. <https://www.oreilly.com/radar/microservices-adoption-in-2020/>. Accessed: 2023-12-20.
- [3] Amr S Abdelfattah, Tomas Cerny, Md Showkat Hossain Chy, Md Arfan Uddin, Samantha Perry, Cameron Brown, Lauren Goodrich, Miguel Hurtado, Muhid Hassan, Yuanfang Cai, et al. 2025. Multivocal study on microservice dependencies. *Journal of Systems and Software* (2025), 112334.
- [4] Amr S. Abdelfattah, Kari E Cordes, Austin Medina, and Tomas Cerny. 2025. Semantic Dependency in Microservice Architecture: A Framework for Definition and Detection. arXiv:2501.11787 [cs.SE] <https://arxiv.org/abs/2501.11787>
- [5] Lauren Adams, Amr S. Abdelfattah, Md Showkat Hossain Chy, Samantha Perry, Patrick Harris, Tomas Cerny, Dario Amoroso d'Aragona, and Davide Taibi. 2024. Evolution and Anti-patterns Visualized: MicroProspect in Microservice Architecture. In *Software Architecture. ECSA 2023 Tracks, Workshops, and Doctoral Symposium*, Bedir Tekinerdoğan, Romina Spalazzese, Hasan Sözer, Silvia Bonfanti, and Danny Weyns (Eds.). Springer Nature Switzerland, Cham, 309–325.
- [6] Dario Amoroso d'Aragona, Alexander Bakhtin, Xiaozhou Li, Ruoyu Su, Lauren Adams, Ernesto Aponte, Francis Boyle, Patrick Boyle, Rachel Koerner, Joseph Lee, et al. 2024. A dataset of microservices-based open-source projects. In *Proceedings of the 21st International Conference on Mining Software Repositories*. 504–509.
- [7] Florian Angerer, Andreas Grimmer, Herbert Prähofer, and Paul Grünbacher. 2019. Change impact analysis for maintenance and evolution of variable software systems. *Automated Software Engineering* 26, 2 (2019), 417–461. doi:10.1007/s10515-019-00253-7
- [8] Alexander Bakhtin, Abdullah Al Maruf, Tomas Cerny, and Davide Taibi. 2022. Survey on tools and techniques detecting microservice api patterns. In *2022 IEEE International Conference on Services Computing (SCC)*. IEEE, 31–38.
- [9] Justus Bogner, Jonas Fritzsche, Stefan Wagner, and Alfred Zimmermann. 2021. Industry practices and challenges for the evolvability assurance of microservices. *Empirical Software Engineering* 26, 5 (2021), 104. doi:10.1007/s10664-021-09999-9
- [10] Vincent Bushong, Amr S. Abdelfattah, Abdullah A. Maruf, Dipta Das, Austin Lehman, Eric Jaroszewski, Michael Coffey, Tomas Cerny, Karel Frajtak, Pavel Tisnovsky, and Miroslav Bures. 2021. On Microservice Analysis and Architecture Evolution: A Systematic Mapping Study. *Applied Sciences* 11, 17 (2021). doi:10.3390/app11177856
- [11] Vincent Bushong, Dipta Das, Abdullah Al Maruf, and Tomas Cerny. 2021. Using static analysis to address microservice architecture reconstruction. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1199–1201.
- [12] Jonida Çarka, Matteo Esposito, and Davide Falessi. 2022. On effort-aware metrics for defect prediction. *Empirical Software Engineering* 27, 6 (2022), 152.
- [13] Tomas Cerny, Amr S. Abdelfattah, Abdullah Al Maruf, Andrea Janes, and Davide Taibi. 2023. Catalog and detection techniques of microservice anti-patterns and bad smells: A tertiary study. *Journal of Systems and Software* (2023).
- [14] Tomas Cerny, Amr S Abdelfattah, Jorge Yero, and Davide Taibi. 2024. From static code analysis to visual models of microservice architecture. *Cluster Computing* (2024), 1–26.
- [15] Tomas Cerny, Md Chy., Amr Abdelfattah., Jacopo Soldani., and Justus Bogner. 2024. On Maintainability and Microservice Dependencies: How Do Changes Propagate?. In *Proceedings of the 14th International Conference on Cloud Computing and Services Science - CLOSER, INSTICC, SciTePress*, 277–286. doi:10.5220/0012725200003711
- [16] Tomas Cerny, Gabriel Goulis Goulis, and Amr Abdelfattah. 2025. Towards Change Impact Analysis in Microservices-based System Evolution. In *2025 IEEE 32nd International Conference on Software Analysis, Evolution and Reengineering (SANER)*. <https://arxiv.org/abs/2501.11778>
- [17] Tomas Cerny, Jan Svacina, Dipta Das, Vincent Bushong, and etc. Walker, Andrew. 2023. Prophet - module for Java-based microservice source code analysis. Retrieved June 27, 2023 from <https://github.com/cloudhubs/prophet-utils>
- [18] Patric Genfer and Uwe Zdun. 2024. Exploring Architectural Evolution in Microservice Systems Using Repository Mining Techniques and Static Code Analysis. In *Software Architecture*, Matthias Galster, Patrizia Scandurra, Tommi Mikkonen, Pablo Oliveira Antonino, Elisa Yumi Nakagawa, and Elena Navarro (Eds.). Springer Nature Switzerland, Cham, 157–173.
- [19] Richard Hutcheson, Austin Blanchard, Jack Hale, Noah Lambaria, David Kozak, Amr Abdelfattah, and Tomas Cerny. 2024. Software Architecture Reconstruction for Microservice Systems Using GraalVM Native Image. In *2024 IEEE 31st International Conference on Software Analysis, Evolution and Reengineering (SANER)*. <https://github.com/cloudhubs/graal-prophet-utils>
- [20] Luka Lelovic, Austin Huzinga, Gabriel Goulis, Anshpreet Kaur, Ricardo Boone, Umidjon Muzrapov, Amr S. Abdelfattah, and Tomas Cerny. 2025. Change impact analysis in microservice systems: A systematic literature review. *Journal of Systems and Software* 219 (2025), 112241. doi:10.1016/j.jss.2024.112241
- [21] Yle News. 2024. Nordea fixes banking bug, warns of more site disruptions over weekend | Yle News | Yle. <https://yle.fi/a/74-20111564>. (Accessed on 12/01/2024).
- [22] Yle News. 2024. Widespread disruption to card payments over | Yle News | Yle. <https://yle.fi/a/74-20075607>. (Accessed on 02/05/2024).
- [23] Garrett Parker, Samuel Kim, Abdullah Al Maruf, Tomas Cerny, Karel Frajtak, Pavel Tisnovsky, and Davide Taibi. 2023. Visualizing anti-patterns in microservices at runtime: A systematic mapping study. *IEEE Access* 11 (2023), 4434–4442.
- [24] Ilaria Pigazzini, Francesca Arcelli Fontana, Valentina Lenarduzzi, and Davide Taibi. 2020. Towards microservice smells detection. In *Proceedings of the 3rd International Conference on Technical Debt*. 92–97.
- [25] Micah Schiewe, Jacob Curtis, Vincent Bushong, and Tomas Cerny. 2022. Advancing Static Code Analysis With Language-Agnostic Component Identification. *IEEE Access* 10 (2022), 30743–30761. doi:10.1109/ACCESS.2022.3160485
- [26] Micah Schiewe, Jacob Curtis, Vincent Bushong, and Tomas Cerny. 2023. Prophet - module for language agnostic-based microservice source code analysis. Retrieved June 27, 2023 from <https://github.com/cloudhubs/prophet2>
- [27] Antonin Smid, Ruolin Wang, and Tomas Cerny. 2019. Case study on data communication in microservice architecture. In *Proceedings of the Conference on Research in Adaptive and Convergent Systems*. 261–267.
- [28] Jacopo Soldani, Stefano Forti, Luca Roveroni, and Antonio Brogi. 2024. Explaining Microservices' Cascading Failures From Their Logs. *Software: Practice and Experience* (2024).
- [29] Davide Taibi, Yuanfang Cai, Ingo Weber, Mehdi Mirakhorli, Michael W Godfrey, John T Stough, and Patrizio Pelliccione. 2023. Continuous Alignment Between Software Architecture Design and Development in CI/CD Pipelines. In *Software Architecture: Research Roadmaps for the Community*. Springer, 69–86.
- [30] Rafik Tighilt, Manel Abdellatif, Imen Trabelsi, Loïc Madern, Naoel Moha, and Yann-Gaël Guéhéneuc. 2023. On the maintenance support for microservice-based systems through the specification and the detection of microservice antipatterns. *Journal of Systems and Software* 204 (2023), 111755. doi:10.1016/j.jss.2023.111755
- [31] Andrew Walker, Dipta Das, and Tomas Cerny. 2020. Automated Code-Smell Detection in Microservices Through Static Analysis: A Case Study. *Applied Sciences*. 10:7800, 21 (November 2020). doi:10.3390/app10217800
- [32] Andrew Walker, Dipta Das, and Tomas Cerny. 2023. MSANose smell detection for microservice systems. Retrieved June 27, 2023 from <https://github.com/cloudhubs/msa-nose>
- [33] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, and Björn Regnell. 2012. *Experimentation in Software Engineering*. Springer.